

BIAI Project Report

Image Classification of Vehicle Brands using Deep Learning

Mateusz Smuda

Year 2026, Semester 6

Contents

1	Introduction of the project topic	2
2	Analysis of the task	2
2.1	Possible approaches to solve the problem	2
2.2	Presentation and description of datasets	3
2.3	Analysis of tools,frameworks and libraries for task solution	3
3	Internal and external specification of the software solution	4
3.1	Classes/objects/methods/scripts/functions	4
3.2	Data structures	4
3.3	User interface	5
4	Experiments	5
4.1	Experimental background	5
4.2	Presentation of experiments with detailed analysis	6
5	Summary	15
6	References	16
7	Links	16

1 Introduction of the project topic

The primary objective of this project is to develop, train, and evaluate a robust Deep Learning model capable of accurately classifying images into specific, visually similar categories of car brands. In recent years, fine-grained image classification has become a critical component of Intelligent Transportation Systems (ITS), autonomous driving, and smart city infrastructure. However, it remains a complex challenge, as vehicles of different brands often share highly similar silhouettes, aerodynamic shapes, and structural components.

Developing this solution allows for the automation of vehicle recognition and brand categorization in real-world scenarios. By exploring various architectures—from lightweight Convolutional Neural Networks (CNNs) to state-of-the-art Vision Transformers (ViT) and advanced Metric Learning techniques—this project aims to prove the effectiveness of modern neural networks in extracting deep, hierarchical visual features to distinguish subtle inter-class differences.

2 Analysis of the task

2.1 Possible approaches to solve the problem

To solve the problem of fine-grained vehicle classification, several approaches were evaluated:

- **Custom CNNs built from scratch:**

Pros: Complete control over the network architecture, low computational overhead, and ease of deployment.

Cons: Simple CNN models typically lack the depth required to extract the complex, hierarchical visual features needed to distinguish visually similar vehicles, often resulting in severe underfitting.

- **Transfer Learning with standard classification:**

Pros: Utilizes pre-trained weights (e.g., ImageNet) to drastically reduce training time and improve baseline accuracy.

Cons: Standard Cross-Entropy loss evaluates predictions independently and often fails to effectively separate visually similar classes in the latent space (e.g., distinguishing a generic sedan made by Honda from one made by Toyota).

- **Metric Learning (Triplet Loss Paradigm):**

Pros: Actively optimizes the latent space by pulling features of the same class together while explicitly pushing features of different classes apart, creating highly distinct boundaries between brands.

Cons: The process of mining positive and negative image pairs can cause severe computational bottlenecks and memory overhead during training if not optimized properly.

Selected Methodology: The project relies primarily on **Transfer Learning** combined with **Metric Learning**. Deep Convolutional Neural Networks (ResNet50) and modern Vision Transformers (ViT) were utilized as backbone feature extractors. To achieve high accuracy on visually similar objects, a dual-loss approach was implemented.

This involves calculating a standard Cross-Entropy Loss for basic classification, coupled with a Cosine Embedding Loss acting as a pseudo-Triplet Loss to analyze and distance fine visual details in the feature vector space.

2.2 Presentation and description of datasets

Two primary datasets were evaluated for training and validation:

- **Car Brands Images (100 Images of Top 50 Car Brands):** A smaller, lightweight dataset consisting of 4,500 car images divided into 50 brand folders. While highly useful for rapid prototyping and initial model debugging, models trained strictly on this dataset suffered heavily from the "Domain Shift" phenomenon. They learned specific studio lighting and angles but failed when tested on raw, unconstrained internet images.
- **Cars Collection Dataset:** A massive, diverse collection consisting of over 60,000 car images across 55 global brands. It includes vehicles in various environments, lighting conditions, and angles, providing the variance needed for real-world application robustness.

Chosen Strategy: Both datasets were merged to create a comprehensive, highly diverse database. To ensure the models did not develop a bias toward statistically dominant car brands, techniques such as the `RandomOverSampler` were evaluated for the Vision Transformer pipeline to guarantee a perfectly balanced class distribution prior to training.

2.3 Analysis of tools, frameworks and libraries for task solution

The software solution was implemented entirely in **Python 3**. The following tools and frameworks were selected based on their performance and industry standards:

- **PyTorch:** The primary deep learning framework chosen for this project. Unlike TensorFlow's static graphs, PyTorch's dynamic computational graph allowed for granular control over the custom data loading pipeline and complex dual-loss functions.
- **TensorFlow / Keras:** Utilized temporarily for the rapid deployment of a MobileNetV2 prototype, specifically leveraging the `tf.data.AUTOTUNE` API for highly optimized, multi-threaded real-time data streaming.
- **timm (Torch Image Models):** An essential library used to instantly instantiate the ResNet50 architecture with the latest pre-trained ImageNet weights, bypassing the need to manually construct deep residual blocks.
- **Hugging Face (Transformers & Datasets):** Leveraged to load, augment, fine-tune, and evaluate the Vision Transformer (`google/vit-base-patch16-224-in21k`).
- **Hardware Environment:** Model training was executed using cloud-based accelerators Google Colab including GPU T4 and A100 with more RAM to handle the intense matrix multiplications required by metric learning.

3 Internal and external specification of the software solution

3.1 Classes/objects/methods/scripts/functions

The internal architecture relies heavily on custom object-oriented structures designed to interface seamlessly with the PyTorch ecosystem.

Key Classes:

- **CustomDataset:** A core class inheriting from `torch.utils.data.Dataset`. Its constructor (`__init__`) scans provided root directories, compiles valid image paths, and dynamically builds mappings between string labels and integer IDs. It overrides the `__getitem__` method to return a comprehensive dictionary containing the query image, its ground-truth label, and the corresponding positive/negative images required for the Triplet Loss logic.
- **SquarePad:** A custom callable class integrated into the `torchvision.transforms` pipeline. Directly resizing rectangular car images distorts vehicle proportions (aspect ratio). **SquarePad** calculates the maximum dimension of an image and applies symmetric zero-padding, creating a perfect square before downscaling it to 224×224 pixels.

Key Methods:

- `get_cls_name(self, path)`: A robust string parsing method that extracts the raw brand name from directory structures or file names. It utilizes a sanitization dictionary (`slownik_naprawczy`) to unify inconsistent naming conventions (e.g., dynamically converting 'MINI' to 'Mini').
- `get_pos_neg_im_paths(self, qry_label)`: The core anchor-mining function. It takes the label of the current query image and pseudo-randomly selects one positive image (matching the brand) and one negative image (a strictly different brand) to form the triplet.
- `get_dls(...)`: A global abstraction function that utilizes `random_split` to divide the dataset into strictly separated subsets (e.g., 80% training, 10% validation, 10% testing), wrapping them in PyTorch **DataLoader** objects configured with specific batch sizes and multi-processing workers.

3.2 Data structures

A significant algorithmic challenge involved the traditional process of mining positive and negative samples for the Triplet Loss function. Iterating through 60,000 files at each training step created an unacceptable CPU bottleneck, ballooning epoch times to over 4 hours.

To resolve this, an optimization was implemented using **Hash Maps (Python Dictionaries)**. During the initialization phase, a specific dictionary named `mapa_klas` is constructed. This data structure pre-groups every file path into arrays mapped to their respective brand keys. Consequently, when `get_pos_neg_im_paths` is executed during training, the time complexity for finding matching and non-matching image pairs

is reduced from $\mathcal{O}(N)$ to $\mathcal{O}(1)$. Furthermore, a primary mapping dictionary (`classes`) is natively exported to a JSON file (`slownik_marek.json`) to guarantee deterministic label-to-integer mapping during future deployment.

3.3 User interface

The current phase of the system operates via a highly verbose, console-based interface.

- **Training Interface:** The `tqdm` library generates rich, color-coded progress bars detailing real-time metrics, including iterations per second (it/s), ongoing loss calculations, and epoch progression.
- **Inference Interface (External Usage):** A dedicated testing script allows users to pass raw images (e.g., JPEG/PNG) to the model. The script loads the saved `.pth` weights, applies the `SquarePad` transform, and processes the tensor. Raw logits are passed through a Softmax activation to compute human-readable probabilities. The interface then prints a formatted terminal report displaying the top 3 most likely vehicle brands alongside their confidence percentages.

4 Experiments

4.1 Experimental background

The project followed a strictly iterative experimental pipeline, gradually increasing structural complexity and dataset volume.

- **Parameters & Conditions Changed:** 1. Model architecture (Simple CNN $\rightarrow$ MobileNetV2 $\rightarrow$ ResNet50 $\rightarrow$ ViT). 2. Dataset volume (4,500 images $\rightarrow$ 60,000 images). 3. Optimization criteria (Standard Cross-Entropy $\rightarrow$ Dual-loss Metric Learning).
- **Purpose of Changes:** These parameters were systematically altered to prove that shallow models fail at fine-grained feature extraction, and to empirically observe how massive data expansion combined with targeted distance-based loss functions mitigates the Domain Shift problem.
- **Calculations & Result Processing:** Raw model outputs (logits) were converted to class predictions using the `argmax` function. Performance was quantified using standard classification accuracy:

$$Accuracy = \frac{TruePositives + TrueNegatives}{TotalObservations}$$

For the metric learning approach, the custom dual-loss function was calculated mathematically as follows:

$$Loss = Loss_{CE}(logits, gt) + Loss_{Cosine}(features_{qry}, features_{pos/neg})$$

This ensures the model is penalized not only for guessing the wrong class but also if the latent feature vectors of similar brands are mathematically too close to each other.

- **Method of Presentation:** Model performance is visually represented through line graphs comparing Epochs against Accuracy/Loss metrics. Additionally, Heatmap Confusion Matrices are utilized to present actual versus predicted classes, visually isolating specific inter-brand confusions (e.g., confusing Hyundai with Honda).

4.2 Presentation of experiments with detailed analysis

Experiment 1: Baseline CNN (PyTorch)

Setup: A simple, three-layer convolutional neural network designed from scratch without pre-trained weights. The model was trained over 6 epochs to establish a baseline performance metric.

Result: As shown in Figure 1, the training process revealed a textbook case of severe overfitting. While the training accuracy rapidly skyrocketed to 98.0% by the final epoch, the validation accuracy (performance on unseen images) stagnated, peaking at a mere 9.8%.

Training Results

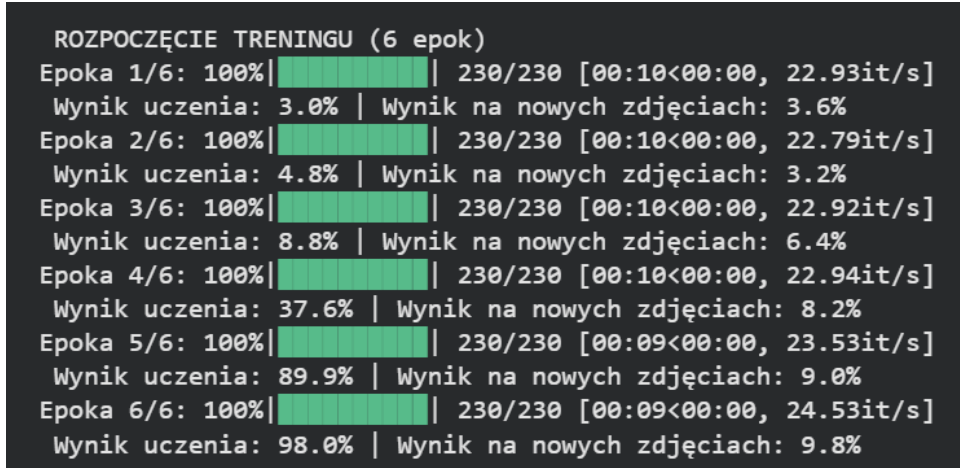


Figure 1: Console output during the training of the Baseline CNN model, illustrating rapid overfitting and a massive discrepancy between training and validation accuracy.

To further analyze this failure, a confusion matrix was generated (Figure 2). A successful classification model typically presents a strong, dark diagonal line (True Positives). In contrast, the baseline CNN's matrix is highly scattered. The model failed to cluster features effectively, often defaulting to random guesses or showing bias toward arbitrary classes (e.g., falsely predicting the 'Fiat' or 'Hudson' classes).

Macierz Pomyłek



Conclusion: Basic, shallow architectures inherently lack the capacity to extract the deep, abstract hierarchical features required to distinguish complex car silhouettes. Instead of learning generalizable features, the model simply memorized the training dataset.

Experiment 2: MobileNetV2 (TensorFlow)

Result: The training halted at epoch 24, achieving a training accuracy of 71.42% and a validation accuracy plateauing around 34.49% (with a final unseen data evaluation score of 32.21%), as shown in Figure 3.

Training Results

Epoch 15/50	
115/115	4s 34ms/step - accuracy: 0.4869 - loss: 1.8476 - val_accuracy: 0.3134 - val_loss: 3.0245
Epoch 16/50	
115/115	5s 37ms/step - accuracy: 0.5171 - loss: 1.7104 - val_accuracy: 0.3221 - val_loss: 3.0122
Epoch 17/50	
115/115	4s 34ms/step - accuracy: 0.5362 - loss: 1.6252 - val_accuracy: 0.3232 - val_loss: 3.0497
Epoch 18/50	
115/115	4s 36ms/step - accuracy: 0.5704 - loss: 1.5208 - val_accuracy: 0.3286 - val_loss: 2.9814
Epoch 19/50	
115/115	4s 36ms/step - accuracy: 0.5919 - loss: 1.4210 - val_accuracy: 0.3286 - val_loss: 3.0370
Epoch 20/50	
115/115	4s 35ms/step - accuracy: 0.6177 - loss: 1.3135 - val_accuracy: 0.3221 - val_loss: 2.9625
Epoch 21/50	
115/115	4s 34ms/step - accuracy: 0.6395 - loss: 1.2526 - val_accuracy: 0.3243 - val_loss: 2.9807
Epoch 22/50	
115/115	4s 36ms/step - accuracy: 0.6737 - loss: 1.1220 - val_accuracy: 0.3482 - val_loss: 3.0255
Epoch 23/50	
115/115	4s 35ms/step - accuracy: 0.6873 - loss: 1.0872 - val_accuracy: 0.3493 - val_loss: 3.0834
Epoch 24/50	
115/115	4s 35ms/step - accuracy: 0.7142 - loss: 1.0022 - val_accuracy: 0.3449 - val_loss: 3.0532
Egzamin końcowy...	
29/29	1s 27ms/step - accuracy: 0.3221 - loss: 2.9625
Ostateczna skuteczność na nowych zdjęciach: 32.21%	

Figure 3: Terminal output from the final epochs of the MobileNetV2 training, showing the ultimate accuracy scores before Early Stopping intervened.

To better visualize the learning dynamics, the training and validation metrics were plotted over time (Figure 4). The learning curves clearly illustrate that while transfer learning provided a massive initial boost compared to the baseline CNN, the model still succumbed to overfitting. Around epoch 5, the validation loss (*Błąd Modelu*) stops decreasing and begins to stagnate, whereas the training loss continues to drop sharply towards 1.0. Simultaneously, the validation accuracy fails to climb above the 35% threshold.

Learning Curves

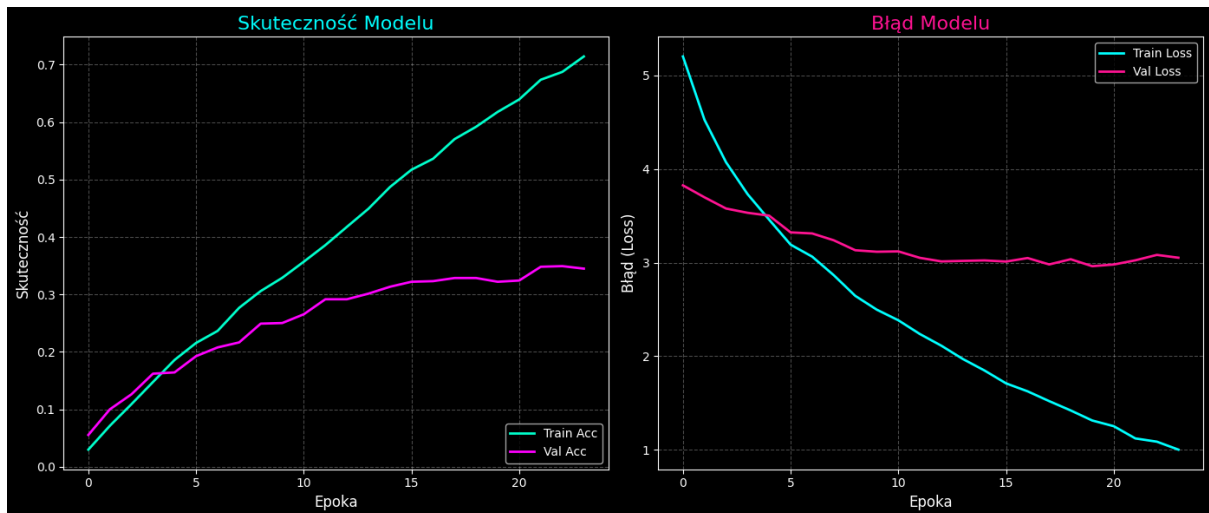


Figure 4: Learning curves for the MobileNetV2 model. The growing divergence between the training (cyan) and validation (magenta) curves perfectly illustrates the network's struggle to generalize fine-grained features.

Conclusion: Transfer learning provides a substantial boost to baseline feature recognition. However, standard lightweight models like MobileNetV2 still lack the parameter

density and deep feature extraction capabilities required to confidently separate over 50 visually similar vehicle classes on a limited dataset.

Experiment 3: ResNet50 with Triplet Loss (Small Dataset)

Setup: Transition to a deep, pre-trained ResNet50 backbone. To address the fine-grained nature of the task, the network was trained using the advanced dual-loss (Cross-Entropy + Cosine Embedding) methodology acting as a Triplet Loss. The model was trained on the smaller 4,500-image dataset over 15 epochs.

Result: The internal metrics were exceptionally high. As shown in Figure 5, the model rapidly converged, achieving 99.5% training accuracy and a peak validation accuracy of 99.3% by epoch 14.

Training Results

```
--- Epoka 13/15 ---
Trening: 100%|██████████| 230/230 [03:29<00:00, 1.10it/s]
Sprawdzanie: 100%|██████████| 29/29 [00:07<00:00, 3.78it/s]
Skuteczność (Z mat. uczył się): 99.5%
Skuteczność (Z mat. nowego): 98.0%
Zapisano nowy najlepszy model.

--- Epoka 14/15 ---
Trening: 100%|██████████| 230/230 [03:29<00:00, 1.10it/s]
Sprawdzanie: 100%|██████████| 29/29 [00:07<00:00, 3.88it/s]
Skuteczność (Z mat. uczył się): 99.4%
Skuteczność (Z mat. nowego): 99.3%
Zapisano nowy najlepszy model.

--- Epoka 15/15 ---
Trening: 100%|██████████| 230/230 [03:29<00:00, 1.10it/s]
Sprawdzanie: 100%|██████████| 29/29 [00:07<00:00, 3.87it/s]Skuteczność (Z mat. uczył się): 99.5%
Skuteczność (Z mat. nowego): 99.1%

Trening zakończony. Najlepszy wynik na walidacji: 99.3%
```

Figure 5: Console output from the final epochs of the ResNet50 training on the small dataset, showcasing near-perfect accuracy metrics.

This phenomenal internal performance is further corroborated by the confusion matrix (Figure 6), which displays a near-perfect, solid diagonal line. Unlike previous models, the ResNet50 with metric learning successfully forced the latent feature vectors of distinct brands apart, resulting in almost zero misclassifications within the validation split.

of 99.41% and a validation accuracy of 99.13%. The minimal gap between training and validation scores proves that the model generalized perfectly without simply memorizing the dataset.

Training Results

```
--- Epoka 13/15 ---
Uczenie sieci: 100%|██████████| 3454/3454 [08:04<00:00, 7.13it/s]
Walidacja: 100%|██████████| 432/432 [00:31<00:00, 13.68it/s]
Dokładność (Trening): 99.29%
Dokładność (Walidacja): 98.57%
Znaleziono lepsze parametry! Zapisuje 'najlepszy_model_pytorch.pth'

--- Epoka 14/15 ---
Uczenie sieci: 100%|██████████| 3454/3454 [08:03<00:00, 7.14it/s]
Walidacja: 100%|██████████| 432/432 [00:32<00:00, 13.38it/s]
Dokładność (Trening): 99.34%
Dokładność (Walidacja): 98.86%
Znaleziono lepsze parametry! Zapisuje 'najlepszy_model_pytorch.pth'

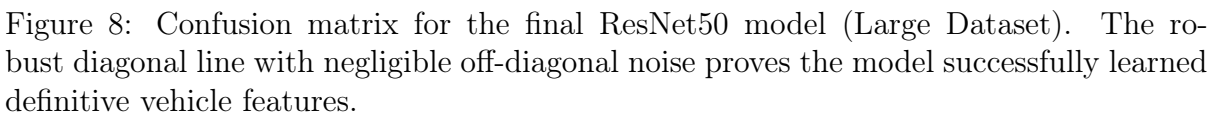
--- Epoka 15/15 ---
Uczenie sieci: 100%|██████████| 3454/3454 [08:04<00:00, 7.12it/s]
Walidacja: 100%|██████████| 432/432 [00:31<00:00, 13.57it/s]
Dokładność (Trening): 99.41%
Dokładność (Walidacja): 99.13%
Znaleziono lepsze parametry! Zapisuje 'najlepszy_model_pytorch.pth'

Proces zakończony. Maksymalna skuteczność walidacyjna: 99.13%
```

Figure 7: Console output from the final epochs of the ResNet50 training on the large dataset. The results indicate stable convergence and excellent generalization capabilities.

The confusion matrix (Figure 8) confirms the model's outstanding performance. The extremely solid, uninterrupted diagonal line across all 55 classes indicates that the Triplet Loss effectively separated the latent features of even the most visually similar brands (e.g., distinguishing a standard Hyundai from a standard Honda) despite the massive variance in background and lighting present in the large dataset.

Macierz Pomyłek - Detekcja błędów w klasyfikacji marek



12

Real-World Inference Test

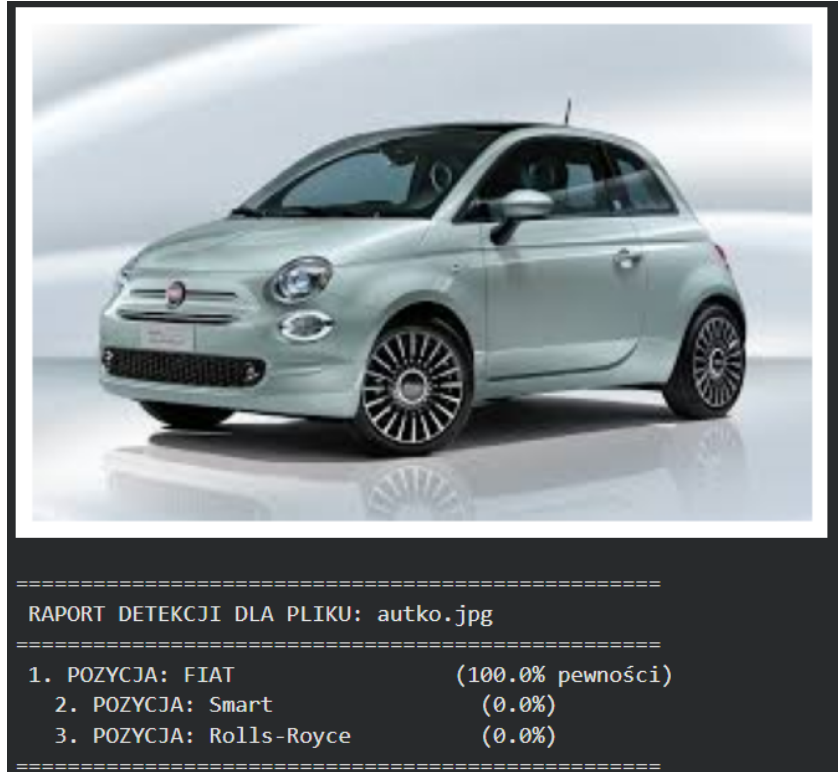


Figure 9: A real-world inference test using a random, unconstrained image downloaded from the internet. The trained ResNet50 model correctly identifies the vehicle with absolute confidence, proving its practical deployment capabilities.

Conclusion: The combination of massive data variance and metric learning successfully neutralized the Domain Shift phenomenon. By exposing the ResNet50 architecture to tens of thousands of diverse images while penalizing poor feature-space distribution, the network was forced to learn actual geometric vehicle features rather than relying on background artifacts. External manual inference on unconstrained internet images was highly successful, confirming this as the definitive solution to the project’s objective.

Experiment 5: Vision Transformer (ViT)

Setup: To explore state-of-the-art Deep Learning paradigms, a Vision Transformer (google/vit-base-p) utilizing the Self-Attention mechanism was implemented. The model’s classification head was modified for the 55 classes. Due to massive computational requirements, training was limited to 6 epochs.

Result: As illustrated in Figure 10, the model achieved a final overall validation accuracy of 91.83%. While numerically lower than the ResNet50 model trained on the large dataset (99.13%), this is an exceptionally strong result considering the architectural shift from convolutions to attention patches and the short training duration.

Training Results

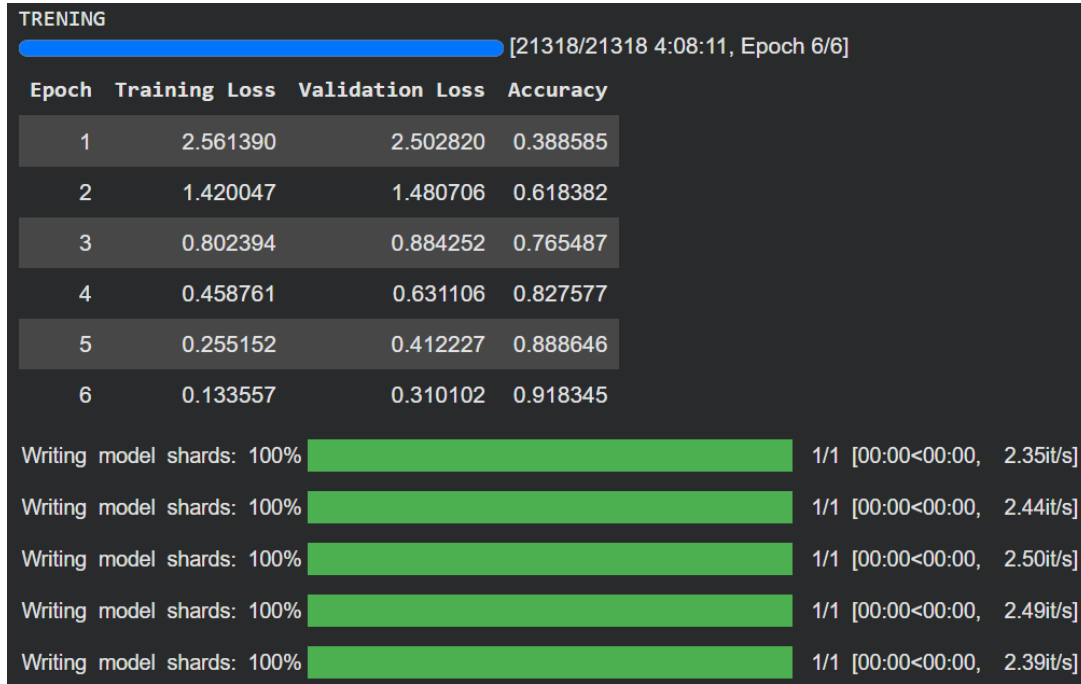


Figure 10: Training and validation loss/accuracy progression over 6 epochs for the Vision Transformer.

A detailed classification report revealed a fascinating insight into how the Transformer perceives vehicles. The model achieved near-perfect precision and recall (1.00 F1-score) on brands with highly distinct, unique design languages (e.g., Bugatti, Lamborghini, Renault, Pontiac). Conversely, the drop in overall accuracy was primarily driven by generic, mass-market brands sharing highly similar silhouettes; for instance, Toyota, Chevrolet, and Ford dropped to approximately 57-62% accuracy. This logic is visually confirmed in the confusion matrix (Figure 11), which maintains a strong diagonal but shows distinct off-diagonal clusters where these generic brands were interchanged.

Confusion Matrix

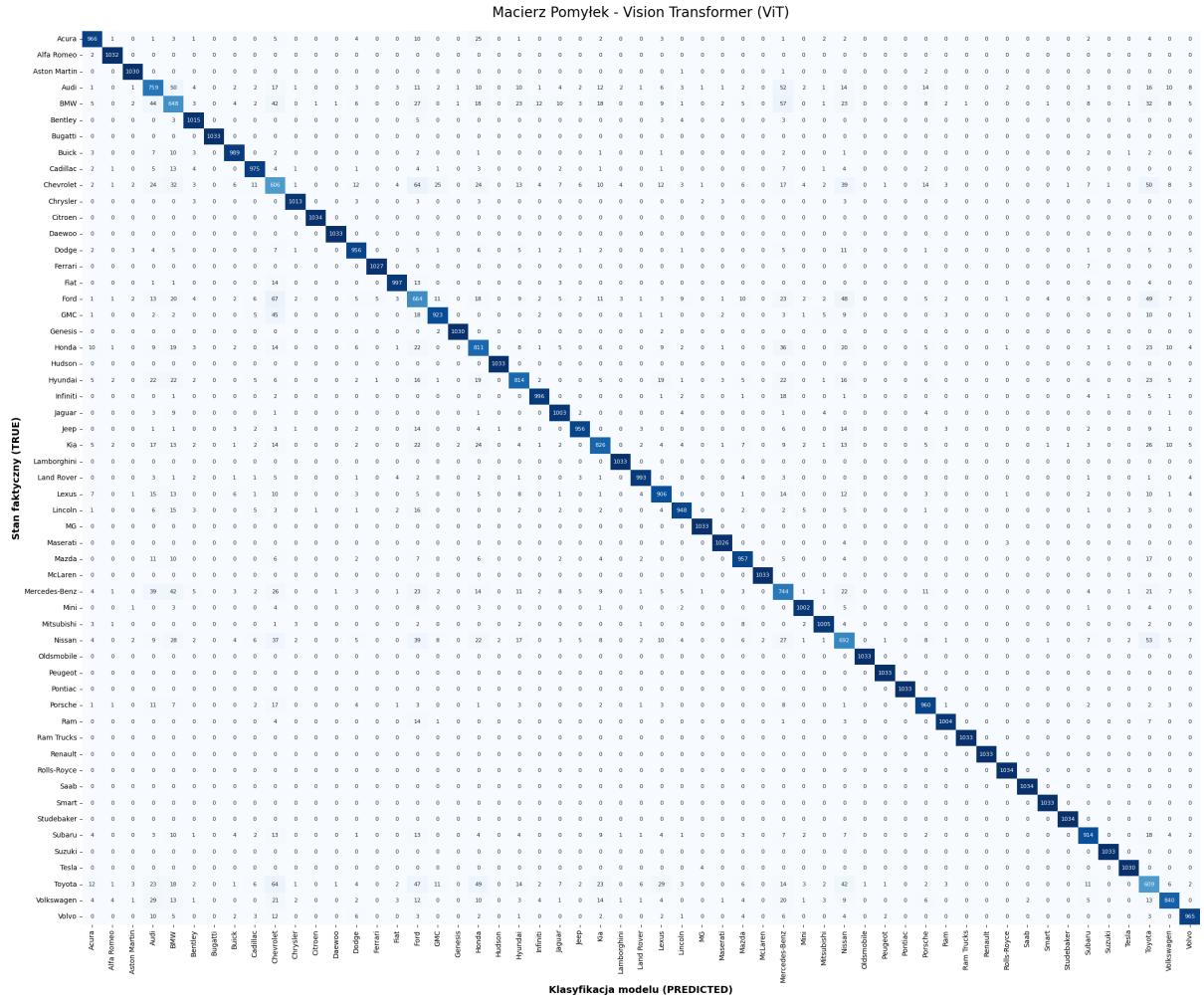


Figure 11: Confusion matrix for the Vision Transformer. The matrix highlights perfect classification for distinct supercars/brands, while revealing clusters of confusion among visually similar, mass-produced vehicles.

Conclusion: Vision Transformers offer highly competitive, context-aware performance compared to deep CNNs. However, for this specific fine-grained visual categorization task, traditional Convolutional Neural Networks (like ResNet50) coupled with explicit distance-based Metric Learning proved slightly superior at extracting the localized, subtle features (like grill patterns or logo shapes) needed to separate highly similar mass-market cars. Consequently, the ResNet50 remains the optimal primary engine for this project's objective.

5 Summary

The project successfully delivered a high-performing, robust classification model capable of distinguishing between 55 visually similar vehicle brands. Empirical experiments proved that shallow networks are insufficient for fine-grained tasks. The integration of deep architectures like ResNet50 utilizing Transfer Learning provided the necessary

feature-extraction capabilities. Furthermore, it was concluded that architectural complexity alone is not enough; neutralizing the Domain Shift phenomenon required a massive expansion of the dataset (to 60,000 images) coupled with advanced Triplet Loss optimization to mathematically separate classes in the latent space.

Future Work and Improvements:

Future iterations of this project will focus on transitioning the software from a console-based script to a fully deployed API. This will serve as the backend for a web-based GUI where users can drag and drop images for real-time classification. Additionally, further exploration of model explainability is planned. Integrating tools like Grad-CAM will allow for the generation of heatmaps, visually highlighting exactly which vehicle components (e.g., grills, headlights, logos) trigger specific brand classifications, thereby increasing the transparency of the neural network.

6 References

References

- [1] *Car Brands Images - 100 Images of Top 50 Car Brands*. Kaggle Dataset.
Available at: <https://www.kaggle.com/datasets/yamaerenay/100-images-of-top-50-car-brands>
- [2] *Cars Collection Dataset*. Kaggle Dataset.
Available at: <https://www.kaggle.com/datasets/ashfaqsyed/cars-collection-dataset>
- [3] Google Research. (n.d.). *Google Colaboratory* (Cloud Computing Environment).
Available at: <https://colab.research.google.com/>

7 Links

All project files, raw code scripts, datasets, and trained model weights are located and organized in the shared clouds and repository:

<https://drive.google.com/drive/folders/1uVkFkIVQNBLPuaw9uw9IjtJXWEK2NEvW?usp=sharing> https://github.com/Mafiash/BIAI_Car_Brands